

Return multiple files from fastapi

Asked 4 years, 1 month ago Modified 5 months ago Viewed 23k times



Using [fastapi](#), I can't figure out how to send multiple files as a response. For example, to send a single file, I'll use something like this

11



```
from fastapi import FastAPI, Response

app = FastAPI()

@app.get("/image_from_id/")
async def image_from_id(image_id: int):

    # Get image from the database
    img = ...
    return Response(content=img, media_type="application/png")
```

However, I'm not sure what it looks like to send a list of images. Ideally, I'd like to do something like this:

```
@app.get("/images_from_ids/")
async def image_from_id(image_ids: List[int]):

    # Get a list of images from the database
    images = ...
    return Response(content=images, media_type="multipart/form-data")
```

However, this returns the error

```
def render(self, content: typing.Any) -> bytes:
    if content is None:
        return b""
    if isinstance(content, bytes):
        return content
    > return content.encode(self.charset)
E AttributeError: 'list' object has no attribute 'encode'
```

python

fastapi

starlette

[Share](#) [Edit](#) [Follow](#)

edited Jan 3 at 10:00



[Brian Tompsett - 汤莱恩](#)

5,827 72 60 132

asked Apr 11, 2020 at 20:04



[Hooked](#)

86.8k 46 195 264

Not sure, but if `content` is a type of `List` then loop content: `for c in content: c.encode() ...`

– felipsmartins Apr 11, 2020 at 20:08

@felipsmartins the objects in the list are bytes already, running `img.encode()` on them doesn't work

'bytes' object has no attribute 'encode' – Hooked Apr 11, 2020 at 20:24

3 Answers

Sorted by: Highest score (default)



11

I've got some problems with @kia's answer on Python3 and latest fastapi so here is a fix that I got working it includes BytesIO instead of StringIO, fixes for response attribute and removal of top level archive folder



```
import os
import zipfile
import io

def zipfiles(filenamees):
    zip_filename = "archive.zip"

    s = io.BytesIO()
    zf = zipfile.ZipFile(s, "w")

    for fpath in filenamees:
        # Calculate path for file in zip
        fdir, fname = os.path.split(fpath)

        # Add file, at correct path
        zf.write(fpath, fname)

    # Must close zip for all contents to be written
    zf.close()

    # Grab ZIP file from in-memory, make response with correct MIME-type
    resp = Response(s.getvalue(), media_type="application/x-zip-compressed",
headers={
        'Content-Disposition': f'attachment;filename={zip_filename}'
    })

    return resp

@app.get("/image_from_id/")
async def image_from_id(image_id: int):

    # Get image from the database
    img = ...
    return zipfiles(img)
```

Share Edit Follow

edited Feb 4, 2022 at 11:12

answered Mar 5, 2021 at 15:19



vozman

1,348 1 16 24



11



Furthermore, you can create the zip on-the-fly and stream it back to the user using a `StreamingResponse` object:

```
import os
import zipfile
import io
from fastapi.responses import StreamingResponse

zip_subdir = "/some_local_path/of_files_to_compress"

def zipfile(filenamees):
    zip_io = io.BytesIO()
    with zipfile.ZipFile(zip_io, mode='w', compression=zipfile.ZIP_DEFLATED) as
temp_zip:
        for fpath in filenamees:
            # Calculate path for file in zip
            fdir, fname = os.path.split(fpath)
            zip_path = os.path.join(zip_subdir, fname)
            # Add file, at correct path
            temp_zip.write(fpath, zip_path)
    return StreamingResponse(
        iter([zip_io.getvalue()]),
        media_type="application/x-zip-compressed",
        headers = { "Content-Disposition": f"attachment; filename=images.zip"}
    )
```

Share Edit Follow

edited Jan 2 at 18:34



KCC13
5 2

answered Jul 5, 2021 at 8:56



shluvme
833 9 26

1 please correct to `temp_zip.write(fpath, zip_path)` – Radoslaw Jan 27, 2022 at 13:51

1 also `zip_subdir = "archive"` here – Gaurav Hazra Sep 6, 2022 at 19:26



10



Zippping is the best option that will have same results on all browsers. you can zip files dynamically.

```
import os
import zipfile
import StringIO

def zipfiles(filenamees):
    zip_subdir = "archive"
    zip_filename = "%s.zip" % zip_subdir

    # Open StringIO to grab in-memory ZIP contents
    s = StringIO.StringIO()
    # The zip compressor
    zf = zipfile.ZipFile(s, "w")

    for fpath in filenamees:
```

```

# Calculate path for file in zip
fdir, fname = os.path.split(fpath)
zip_path = os.path.join(zip_subdir, fname)

# Add file, at correct path
zf.write(fpath, zip_path)

# Must close zip for all contents to be written
zf.close()

# Grab ZIP file from in-memory, make response with correct MIME-type
resp = Response(s.getvalue(), mimetype = "application/x-zip-compressed")
# ..and correct content-disposition
resp['Content-Disposition'] = 'attachment; filename=%s' % zip_filename

return resp

@app.get("/image_from_id/")
async def image_from_id(image_id: int):

    # Get image from the database
    img = ...
    return zipfiles(img)

```

As alternative you can use base64 encoding to embed an (very small) image into json response. but i don't recommend it.

You can also use MIME/multipart but keep in mind that i was created for email messages and/or POST transmission to the HTTP server. It was never intended to be received and parsed on the client side of a HTTP transaction. Some browsers support it, some others don't. (so i think you shouldn't use this either)

Share Edit Follow

edited Apr 12, 2020 at 1:10

answered Apr 11, 2020 at 20:15



kia

171

1

7

- 1 Hi @kia, and thank you for the answer! Stack Overflow discourages single link answers (and it may get downvoted as a result). The answer would be improved if you could provide a small example that shows how to use `aiofiles` in this particular context. – [Hooked](#) Apr 11, 2020 at 20:21

Hi @Hooked and @kia, unless I'm missing something, zipping files as in your example is a blocking IO operation that might mess the asynchronous event loop under the hood. Consider either making a synchronous handler (remove the `async` in the definition of `image_for_id`), or consider running the `zipfiles` function under the control of a `ThreadPoolExecutor` as in this example : docs.python.org/3/library/... – [glenfant](#) May 19, 2020 at 16:56

@glenfant Can you please tell me why **"a blocking IO operation might mess the asynchronous event loop under the hood"**? thank you. – [kia](#) May 20, 2020 at 0:02